

Módulo 9

Publicación y Consumo de Datos

Curso: Data Engineer Básico

1.5 horas

Objetivos de Aprendizaje

- **Comprender** los conceptos de reporting y analítica en el contexto de ingeniería de datos
- **Diseñar** contratos de consumo efectivos para APIs y datasets
- **Implementar** servicios API básicos para publicar datos consumibles
- **Aplicar** estándares de documentación para facilitar el consumo de datos por equipos internos

3.1 Reporting y Analítica

Conceptos Fundamentales

¿Qué es el Reporting?

El **reporting** es el proceso de transformar datos procesados en información **actionable** para la toma de decisiones.

- **Periodicidad definida:** Diario, semanal, mensual
- **Audiencia conocida:** Usuarios de negocio
- **Métricas estandarizadas:** KPIs predefinidos
- **Formato consistente:** Misma estructura cada vez

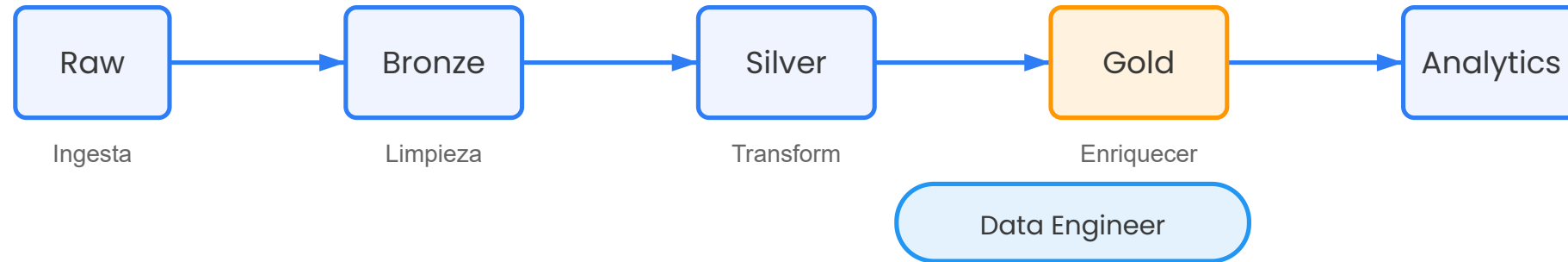
Recuerda: A diferencia del análisis exploratorio, el reporting tiene estructura predecible y repetible.

Tipos de Reporting

Tipo	Descripción	Ejemplo
Operacional	Monitoreo en tiempo real o casi-real	Dashboard de estado de pipelines
Táctico	Análisis de mediano plazo	Reporte semanal de calidad
Estratégico	Visiones de largo plazo	Tendencias trimestrales

El Data Engineer como Intermediario

El Data Engineer conecta los datos crudos con los consumidores analíticos:



Responsabilidades clave:

- Garantizar disponibilidad en capas gold
- Mantener SLAs de actualización
- Asegurar calidad y consistencia de métricas

Métricas vs KPIs

Métrica

Una **medida cuantificable** sin contexto de negocio.

```
total_records_processed = 15000
```

KPI

Métrica con **contexto y umbral objetivo**.

```
data_quality_score > 95%
```

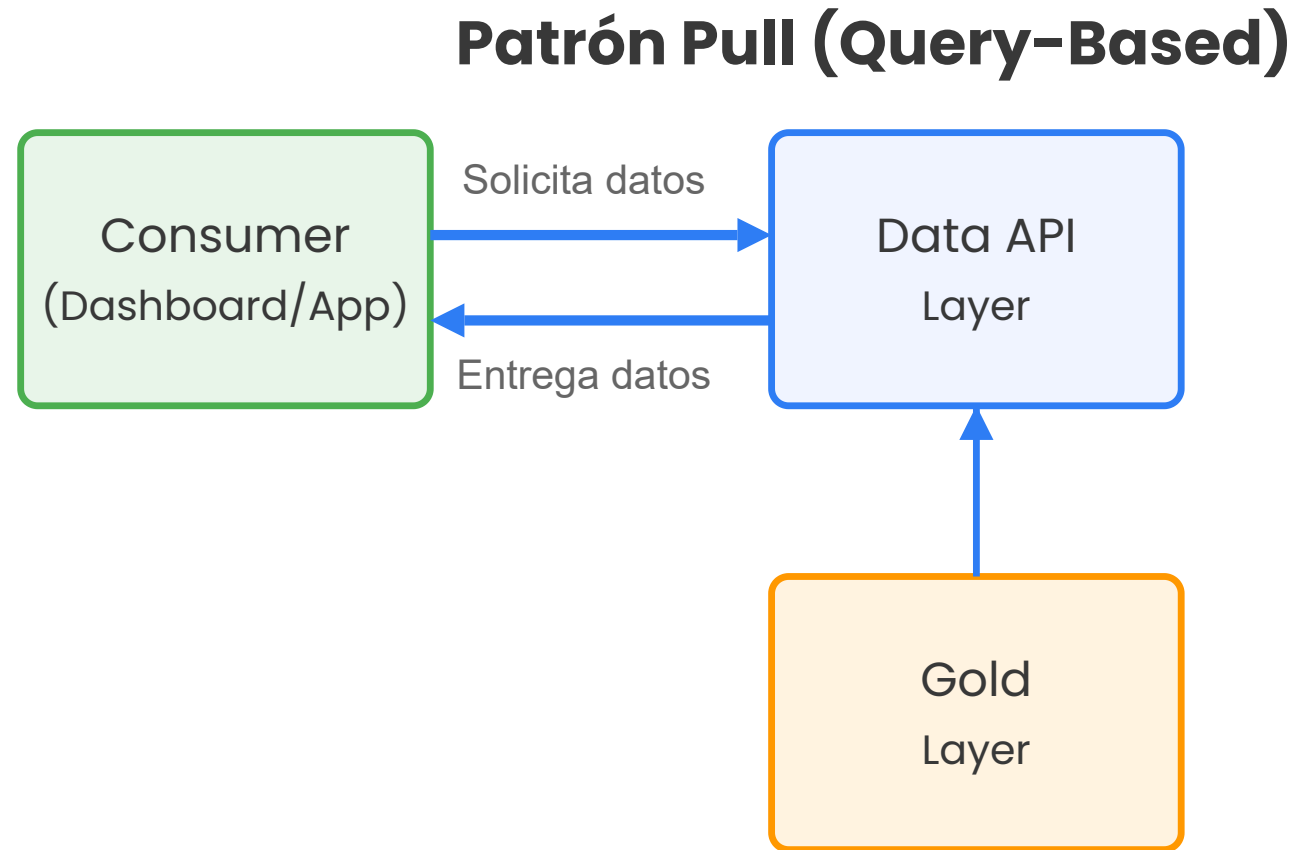
Importante: Un KPI sin definición clara de negocio no es un KPI. Es solo un número.

3.2 Arquitecturas de Publicación

Pull vs Push

Patrón Pull (Query-Based)

El consumidor **solicita datos bajo demanda** cuando los necesita.



El consumidor controla cuándo obtiene los datos

Casos de uso ideales:

- Dashboards con refresco manual
- Análisis ad-hoc
- Datos con baja frecuencia de cambio

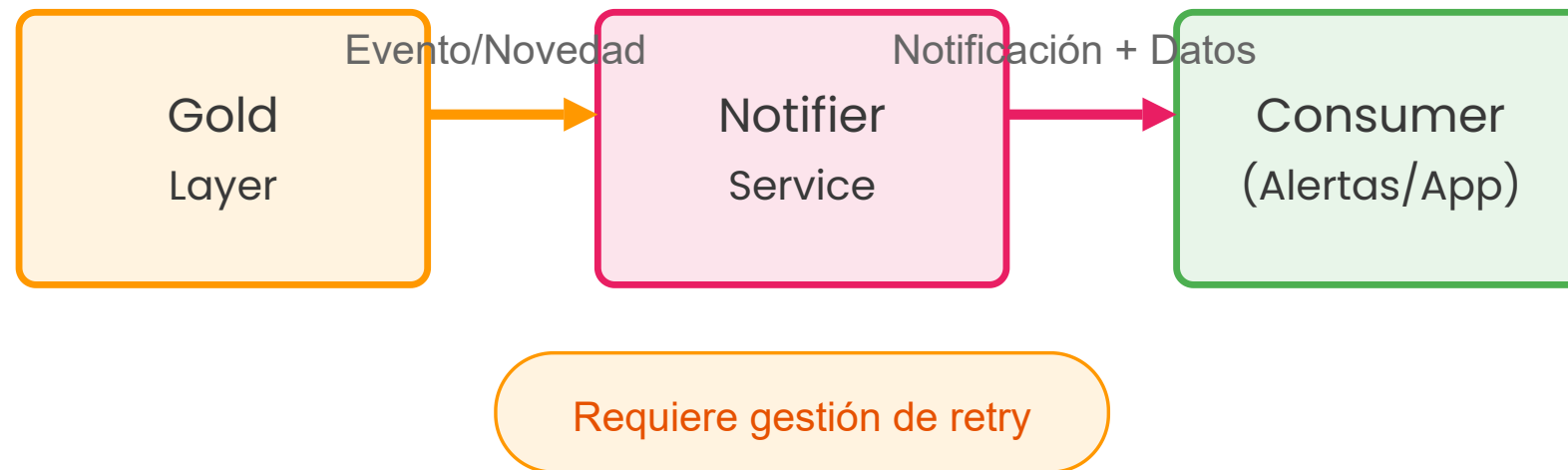
✓ **Ventajas:**

- Consumidor controla cuándo obtiene datos
- No hay costos de almacenamiento adicional
- Easy de versionar

Patrón Push (Event-Based)

El sistema **envía datos a los consumidores** cuando hay novedades.

Patrón Push (Event-Based)



El sistema envía datos cuando hay novedades

Casos de uso ideales:

- Alertas en tiempo real
- Sistemas que requieren datos actualizados
- Integraciones con otros sistemas

⚠ Considerations:

- Mayor complejidad de implementación
- Necesidad de gestionar retry y dead letters

Selección de Patrón

Criterio	Pull	Push
Frecuencia de actualización	Baja-media	Media-alta
Latencia aceptable	Minutos-horas	Segundos-minutos
Complejidad	Baja	Media-alta
Costo de infraestructura	Variable	Fijo + variable

3.3 Contratos de Consumo

Diseño y Versioning

¿Qué es un Contrato de Consumo?

Un **contrato de consumo** define explícitamente **qué datos** están disponibles, **en qué formato**, y **bajo qué condiciones**.



Elementos de un Contrato

```
contrato_consumo:
  nombre: "kpis_mantenimiento_v1"
  version: "1.0.0"
  propietario: "equipo_data"

esquema:
  campos:
    - nombre: "asset_id"
      tipo: "string"
      descripcion: "Identificador único del activo"
      nullable: false
    - nombre: "kpi_availability"
      tipo: "float"
      unidad: "porcentaje"
      rango: [0, 100]

disponibilidad:
  frecuencia_actualizacion: "daily"
  sla_actualizacion: "06:00 UTC"
```


Versionado Semántico

MAJOR.MINOR.PATCH

1.0.0 → 1.1.0 → 2.0.0

MAJOR (2.0.0)

Cambios que rompen compatibilidad

MINOR (1.1.0)

Nuevos campos opcionales

PATCH (1.0.1)

Bug fixes

Principios Clave

Inmutabilidad de Versiones

Una vez publicada una versión, **nunca** se modifican los campos existentes.

Solo se añaden nuevos campos opcionales.

Deprecación

```
deprecacion:  
  version_afectada: "1.2.0"  
  fecha_deprecacion: "2026-06-01"  
  alternativa: "kpis_v2"
```

Case Study: Contrato TransCore

```
# Contrato: kpis_mantenimiento_transcore_v1
contrato:
  nombre: "kpis_mantenimiento_transcore"
  version: "1.0.0"
  dominio: "mantenimiento"

esquema:
  - campo: "work_order_id"
    tipo: "string"
    ejemplo: "WO-2026-0415-001"
  - campo: "failure_probability"
    tipo: "float"
    unidad: "porcentaje"
    rango: [0, 100]
  - campo: "recommended_next_service_date"
    tipo: "date"
  - campo: "calculated_at"
    tipo: "timestamp"
```

3.4 Servicios API para Datos

Fundamentos de REST API

Conceptos Fundamentales

Concepto	Descripción
Endpoint	URL específica que representa un recurso
Método HTTP	Operación a realizar (GET, POST, PUT, DELETE)
Request	Datos enviados por el cliente al servidor
Response	Datos devueltos por el servidor al cliente
Status Code	Código que indica el resultado de la operación

Métodos HTTP Comunes

Método	Propósito	Ejemplo
GET	Obtener datos	GET /api/v1/kpis/mantenimiento
POST	Crear nuevo recurso	POST /api/v1/suscripciones
PUT	Actualizar recurso existente	PUT /api/v1/kpis/123
DELETE	Eliminar recurso	DELETE /api/v1/cache/datos-antiguos

Diseño de Endpoints

```
# Estructura común de endpoints para datasets
```

```
GET /api/v1/{dominio}          # Listar todos los registros
GET /api/v1/{dominio}/{id}     # Obtener un registro específico
GET /api/v1/{dominio}?param=value # Filtrar resultados
```

```
# Ejemplos concretos
```

```
GET /api/v1/kpis/mantenimiento # Todos los KPIs
GET /api/v1/kpis/mantenimiento/WO-2026-001 # KPI específico
GET /api/v1/kpis/mantenimiento?asset=AST-* # Filtrar por asset
```

Códigos de Estado HTTP

Código	Significado
200	OK - Request exitosa
201	Created - Recurso creado
400	Bad Request - Formato inválido
401	Unauthorized - Falta autenticación
404	Not Found - Recurso no existe
429	Too Many Requests - Rate limit
500	Internal Server Error

Tip: Los códigos 2xx indican éxito, 4xx errores del cliente, y 5xx errores del servidor.

Ejemplo de Response JSON

```
{
  "status": "success",
  "data": {
    "work_order_id": "WO-2026-0415-001",
    "asset_id": "AST-MOTOR-001",
    "failure_probability": 23.5,
    "recommended_next_service_date": "2026-05-15",
    "calculated_at": "2026-04-21T05:30:00Z"
  },
  "meta": {
    "version": "1.0.0",
    "generated_at": "2026-04-21T05:30:01Z"
  }
}
```

3.5 Documentación y Catálogo

Hace tus datos descubribles y comprensibles

Niveles de Documentación

Nivel	Descripción	Audiencia
Técnica	Esquemas, tipos de datos, APIs	Data Engineers, Developers
Semántica	Significado de campos, cálculos	Analistas, Data Scientists
Negocio	Definiciones de KPIs, contexto	Usuarios de negocio

Descripciones de Campos

✗ Mal: Sin contexto

```
campo: "valor"
```

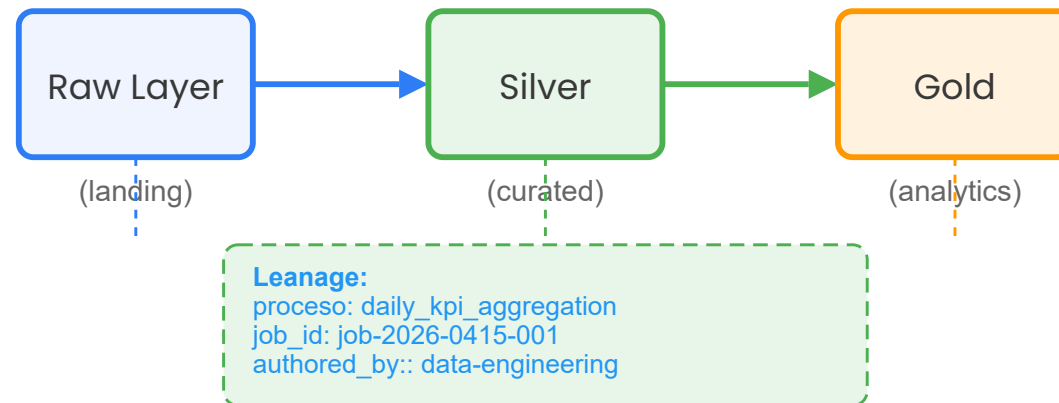
✓ Bien: Con descripción

```
campo:  
  nombre: "failure_probability"  
  descripcion: "Probabilidad estimada de fallo en 30 días"  
  tipo: "float"  
  unidad: "porcentaje"  
  ejemplo: 23.5  
  reglas_negocio: "Modelo XGBoost con datos históricos"
```

Metadatos de Calidad

```
calidad_datos:  
  - dataset: "gold_kpis_mantenimiento"  
  freshness:  
    ultima_actualizacion: "2026-04-21T05:30:00Z"  
    sla_cumplido: true  
  completeness:  
    registros_totales: 1523  
    porcentaje: "99.7%"  
  accuracy:  
    validado_contra: "sistema_fuente"  
    resultado: "PASS"
```

Linaje de Datos



```
lineage:  
  proceso: "daily_kpi_aggregation"  
  job_id: "job-2026-0415-001"  
  authored_by: "data-engineering"
```

Herramientas de Catálogo

Herramienta	Tipo	Uso común
DataHub	Open Source	Metadata centralizado, lineage
Apache Atlas	Open Source	Governance y compliance
Collibra	Comercial	Data catalog enterprise
Azure Purview	Cloud	Integración ecosistema Microsoft
Dataplex	Cloud	Governance en Google Cloud

Mini-reto

Definir Contrato de Consumo para Indicador Crítico

Contexto TransCore

El equipo de operaciones necesita acceder diariamente a un KPI crítico: "**Tiempo promedio de resolución de incidencias**".

Objetivo: Diseñar el contrato de consumo que defina cómo los equipos de operaciones accederán a este KPI.

Preguntas a considerar:

- ¿Qué campos son esenciales?
- ¿Con qué frecuencia necesitan el dato?
- ¿Quiénes son los consumidores autorizados?

Criterios de evaluación:

- Esquema completo (4+ campos)
- Descripciones claras
- SLA realista
- Consideración de errores

Plantilla de Entrega

```
contrato:
  nombre: "incidente_resolucion_kpi"
  version: "1.0.0"
  dominio: "operaciones"

esquema:
  campos:
    # Definir al menos 4 campos con nombre, tipo, descripción

disponibilidad:
  frecuencia_actualizacion: "" # daily, hourly, etc.
  sla_actualizacion: "" # ej: "06:00 UTC"

acceso:
  nivel: "" # interno, público, etc.
  equipos_autorizados: []

contacto:
  owner: "" # email del responsable
```

Criterios de Evaluación

Criterio	Puntos
Esquema completo con al menos 4 campos	25
Descripciones claras y negocio-friendly	25
SLA realista y alcanzable	20
Consideración de casos de error	15
Versionado semántico correcto	15
Total	100

Duración: 15 minutos

Resumen

Conceptos Clave

Concepto	Qué debes recordar
Reporting	Transformar datos en información actionable
Pull vs Push	Pull = bajo demanda; Push = eventos
Contrato de consumo	Define qué, cómo, cuándo y quién
Versionado semántico	MAJOR rompe, MINOR añade, PATCH fija
API REST	Acceso programático mediante endpoints
Documentación	Técnica, semántica y de negocio

Patrones de Diseño

✓ Hacer:

- Usar versionado semántico
- Documentar con ejemplos
- Definir SLAs claros
- Incluir metadatos de calidad

✗ Evitar:

- Cambiar contratos sin versionar
- Olvidar significado de campos
- No definir qué pasa si SLA falla
- Dejar sin especificar rate limiting

Próximos Pasos

- **Diseñar** un contrato de consumo completo
- **Entender** cuándo usar patrones pull vs push
- **Conocer** los elementos de una API REST
- **Practicar** definiendo un contrato real

Referencias

Lecturas Complementarias

Recurso	Descripción
Designing Data-Intensive Applications	Capítulos sobre datos derivados y batch/stream - Martin Kleppmann
DataContract.slashdata	Estándar open source para contratos de datos
RESTful API Design	Best practices para APIs HTTP

Herramientas de Referencia

Herramienta	Uso
Swagger/OpenAPI	Documentación de APIs
DataHub	Catálogo de datos open source
JSON Schema	Validación de esquemas

Glosario

Término	Definición
Contrato de consumo	Acuerdo formal entre proveedor y consumidor de datos
KPI	Key Performance Indicator - Métrica con objetivo
Linaje	Rastro de transformación de datos
SLA	Service Level Agreement - Compromiso de disponibilidad
Rate limit	Límite de requests que un consumidor puede hacer

Gracias!

Módulo 9 Completado

Siguiente: Módulo 10